Name: Arpit Giri

BTech Cse, 3rd year

sec:"B"

Cu24250181

ROll no. = 11

Experiment No. 1

Experiment Name

Real-World Data Collection, Cleaning and Preprocessing using Python

Aim

To collect a real-world dataset from publicly available sources and perform data preprocessing by handling missing values, duplicate records, inconsistent formats, categorical variables, and outliers using Python.

GITHUB LINK: https://github.com/Arpitcu24250181/Mastery-in-Data-Analytics-and-Visualizations-and-Career- Advancement/blob/main/Experiment%201.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
df = sns.load_dataset('titanic')

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

```
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column                    Non-Null Count Dtype
--- ------                    -------------- -----
  0  survived      891 non-null  int64
  1  pclass        891 non-null  int64
  2  sex           891 non-null  object
 3   age           714 non-null  float64
 4   sibsp         891 non-null  int64
  5  parch         891 non-null  int64
  6  fare          891 non-null  float64
  7  embarked      889 non-null  object
  8  class who     891 non-null  category
 9                 891 non-null  object
                   891 non-null  bool
 10 adult_male     203 non-null  category
 11 deck
    12 embark_town 889 non-null object
                   891 non-null  object
 13   alive        891 non-null  bool
 14 alone
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

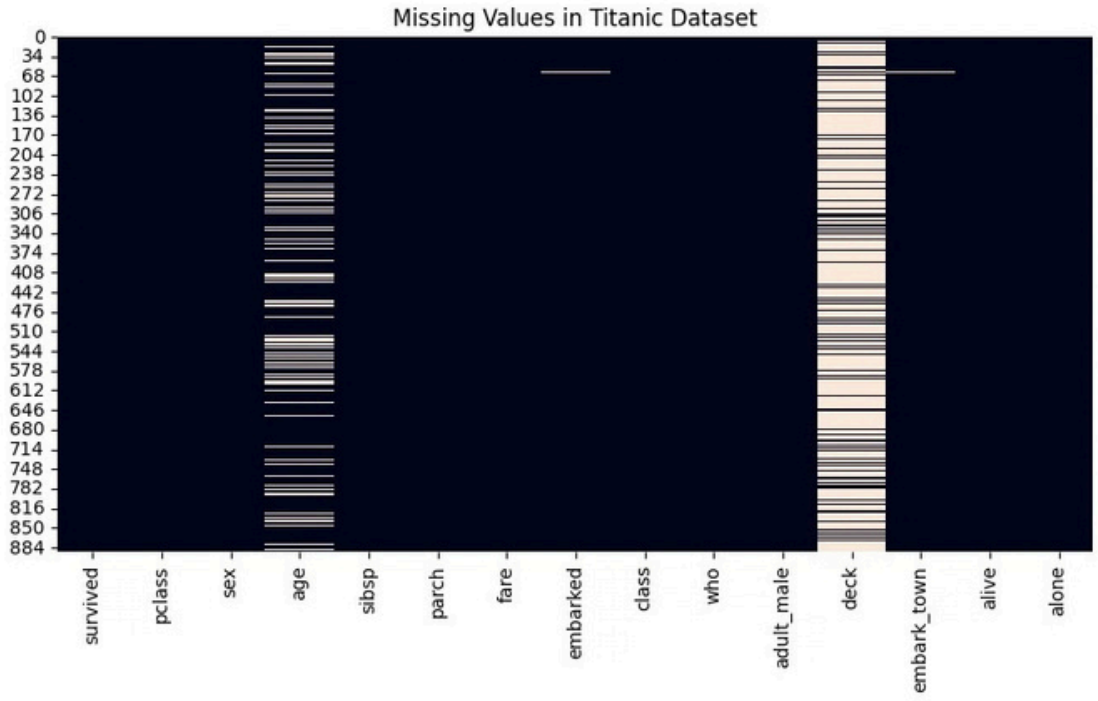
```
df.describe()
```

	survived	pclass	age	sibsp	parch	fare
	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
count	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
mean	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
std	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
min	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
25%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
50%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
75%	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200
max						

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

```
plt.figure(figsize=(10, 5))
sns.heatmap(df.isnull(), cbar=False)
plt.title("Missing Values in Titanic Dataset")
plt.show()
```




```
# Fill missing Age values with median
df['age'] = df['age'].fillna(df['age'].median())

# Fill missing Embarked values with mode
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Fill missing Embark Town values with mode
df['embark_town'] = df['embark_town'].fillna(df['embark_town'].mode()[0])
```

```
print("Missing values after treatment:")
print(df.isnull().sum())
```

```
Missing values after treatment:
survived         0
pclass           0
sex              0
age              0
sibsp            0
parch            0
fare             0
embarked         0
class            0
who              0
adult_male       0
deck           688
embark_town      0
alive            0
alone            0
dtype: int64
```

```
print("Number of duplicate records:", df.duplicated().sum())
```

```
Number of duplicate records: 110
```

```
df = df.drop_duplicates()

print("Duplicate records after removal:", df.duplicated().sum())
```

```
Duplicate records after removal: 0
```

```
categorical_columns = df.select_dtypes(include=['object', 'category']).columns

print("Categorical columns:")
print(categorical_columns)
```

```
Categorical columns:
Index(['sex', 'embarked', 'class', 'who', 'deck', 'embark_town', 'alive'], dtype='object')
```

```
le = LabelEncoder()

df['sex_encoded'] = le.fit_transform(df['sex'])

df[['sex', 'sex_encoded']].head()
```

	sex	sex_encoded
0	male	1
1	female	0
2	female	0
3	female	0
4	male	1

```
df = pd.get_dummies(df, columns=['embarked'], prefix='embarked', dtype=int)

df.head()
```

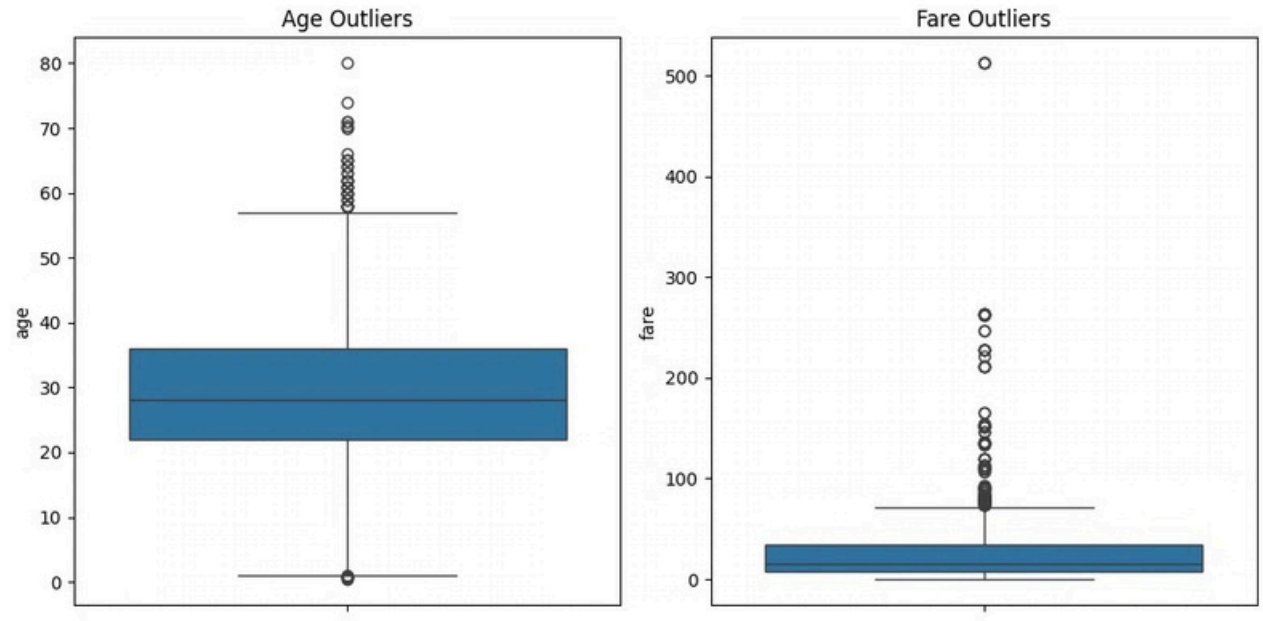
	survived	pclass	sex	age	sibsp	parch	fare	class	who	adult_male	deck	embark_town	alive	alone	sex_encod
0	0	3	male	22.0	1	0	7.2500	Third man	True	NaN	Southampton	no	False		
1	1	1	female	38.0	1	0	71.2833	First woman	False	C	Cherbourg	yes	False		
2	1	3	female	26.0	0	0	7.9250	Third woman	False	NaN	Southampton	yes	True		
3	1	1	female	35.0	1	0	53.1000	First woman	False	C	Southampton	yes	False		
4	0	3	male	35.0	0	0	8.0500	Third man	True	NaN	Southampton	no	True		

```
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
sns.boxplot(y=df['age'])
plt.title("Age Outliers")

plt.subplot(1, 2, 2)
sns.boxplot(y=df['fare'])
plt.title("Fare Outliers")
```

```
plt.tight_layout()
plt.show()
```




```
def detect_outliers(column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = df[(df[column] < lower) | (df[column] > upper)]

    print(f"{column}: {len(outliers)} outliers")
    print(f"Lower      limit:      {lower:.2f}")
    print(f"Upper limit: {upper:.2f}")

detect_outliers('age')
detect_outliers('fare')
```

```
age:   39   outliers
Lower  limit:  1.00
Upper limit: 57.00
fare: 102 outliers
Lower limit: -30.91
Upper limit: 72.98
```

```
def cap_outliers(column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[column] = df[column].clip(lower=lower, upper=upper)

cap_outliers('age')
cap_outliers('fare')

print("Outliers treated successfully.")
```

```
Outliers treated successfully.
```

```
df['family_size'] = df['sibsp'] + df['parch'] + 1

df[['sibsp', 'parch', 'family_size']].head()
```

	sibsp	parch	family_size
0	1	0	2
1	1	0	2
2	0	0	1
3	1	0	2
4	0	0	1

```
df['age_group'] = pd.cut(
    df['age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['Child', 'Teenager', 'Young Adult', 'Adult', 'Senior']
)

df[['age', 'age_group']].head()
```

	age	age_group
0	22.0	Young Adult
1	38.0	Adult
2	26.0	Young Adult
3	35.0	Young Adult
4	35.0	Young Adult

```
scaler = StandardScaler()

df[['age', 'fare']] = scaler.fit_transform(df[['age', 'fare']])

print("Numerical features standardized successfully.")
```

```
Numerical features standardized successfully.
```

```
df[['age', 'fare']].describe()
```

	age	fare
count	7.810000e+02	7.810000e+02
mean	1.182722e-16	-6.141054e-17
std	1.000641e+00	1.000641e+00
min	-2.170008e+00	-1.166698e+00
25%	-5.622425e-01	-8.118984e-01
50%	-1.028809e-01	-4.659132e-01
75%	5.096012e-01	3.327529e-01
max	2.117367e+00	2.049730e+00

```
print("Final dataset shape:", df.shape)

df.head()
```

```
Final dataset shape: (781, 20)
```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	adult_male	deck	embark_town	alive	alone	sex
0	0	3	male	-0.562242	1	0	-0.847158	Third	man	True	NaN	Southampton	no	False	
1	1	1	female	0.662722	1	0	1.975081	First	woman	False	C	Cherbourg	yes	False	
2	1	3	female	-0.256001	0	0	-0.817408	Third	woman	False	NaN	Southampton	yes	True	
3	1	1	female	0.433041	1	0	1.173660	First	woman	False	C	Southampton	yes	False	
4	0	3	male	0.433041	0	0	-0.811898	Third	man	True	NaN	Southampton	no	True	

```
print("Remaining missing values:")
print(df.isnull().sum())
```

```
Remaining missing values:
survived         0
pclass           0
sex              0
age              0
sibsp            0
parch            0
fare             0
class            0
who              0
adult_male       0
deck           579
embark_town      0
alive            0
alone            0
sex_encoded      0
embarked_C       0
embarked_Q       0
embarked_S       0
family_size      0
age_group        0
dtype: int64
```

```
df.to_csv('titanic_cleaned_preprocessed.csv', index=False)

print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

```
from google.colab import files

files.download('titanic_cleaned_preprocessed.csv')
```

Questions and Answers

Q1. Why is data preprocessing considered one of the most important phases in data analytics?

Answer: Data preprocessing is an important phase of data analytics because raw data may contain missing values, duplicate records, inconsistent values, categorical data, and outliers. Preprocessing improves the quality and consistency of data and makes it suitable for analysis and machine learning. Clean and properly prepared data helps in obtaining more accurate and reliable results.

Q2. Explain different methods of handling missing values with suitable examples.

Answer: Missing values can be handled using different methods depending on the type and amount of missing data.

1. **Mean:** Missing numerical values can be replaced with the mean value. *Example:* Missing marks can be replaced by the average marks.
2. **Median:** Missing numerical values can be replaced with the median. It is useful when the data contains outliers. *Example:* Missing age values can be replaced with the median age.
3. **Mode:** Missing categorical values can be replaced with the most frequently occurring value. *Example:* Missing gender or city values can be replaced with the mode.
4. **Removal:** Rows or columns having too many missing values can be removed when appropriate.

Q3. Differentiate between Label Encoding and One-Hot Encoding.

Answer:

Label Encoding converts categories into numerical labels. For example, Male can be represented as 1 and Female as 0.

One-Hot Encoding creates separate binary columns for each category. For example, a Gender column containing Male and Female can be converted into two columns containing 0 and 1 values.

Label Encoding uses fewer columns, while One-Hot Encoding is generally useful for nominal categories because it does not assign an artificial numerical order to them. Scikit-learn provides LabelEncoder and OneHotEncoder for these types of transformations.

Q4. What are outliers? How can they affect analytical results?

Answer: Outliers are data values that are significantly different from the other observations in a dataset. They may occur because of measurement errors, unusual situations, or genuine extreme observations.

Outliers can affect the mean, standard deviation, correlation, and other statistical results. They can also negatively affect some machine learning algorithms. Therefore, outliers should be detected and treated appropriately.

Q5. Explain the difference between normalization and standardization.

Answer:

Normalization scales numerical values to a specific range, commonly from 0 to 1.

Standardization transforms data so that it has a mean close to 0 and a standard deviation close to 1.

Normalization is useful when features need to be within a fixed range, while standardization is commonly used when machine learning algorithms work better with features on a similar scale. Scikit-learn provides tools such as `MinMaxScaler` and `StandardScaler` for these transformations.

Q6. Why should duplicate records be removed before analysis?

Answer: Duplicate records should be removed because they may cause the same observation to be counted multiple times. This can produce biased statistical results and affect the performance of machine learning models. Removing duplicates helps maintain the accuracy and consistency of the dataset.

Pandas provides `duplicated()` to identify duplicate rows and `drop_duplicates()` to remove them.

Q7. What is feature engineering? Give two practical examples.

Answer: Feature engineering is the process of creating new and useful features from existing data to improve data analysis or machine learning performance.

Examples:

1. **Family Size:** In the Titanic dataset, Family Size can be created using: `Family Size = SibSp + Parch + 1`
2. **Age Group:** The Age column can be divided into groups such as Child, Teenager, Adult, and Senior.

Q8. Which preprocessing techniques would you apply to the IBM HR Employee Attrition dataset and why?

Answer: The following preprocessing techniques can be applied to the IBM HR Employee Attrition dataset:

1. **Missing Value Treatment** – to handle incomplete employee records.
2. **Duplicate Removal** – to avoid repeated employee information.
3. **Categorical Encoding** – to convert attributes such as Department, Job Role, and Gender into numerical form.
4. **Outlier Detection** – to identify unusual values in features such as Monthly Income and Total Working Years.
5. **Feature Scaling** – to bring numerical features to a comparable scale.
6. **Feature Engineering** – to create useful features such as Age Group or Income Category.

These techniques improve the quality of the dataset and make it suitable for further analysis and machine learning.

Q9. How does poor-quality data affect machine learning model performance?

Answer: Poor-quality data can reduce the accuracy and reliability of a machine learning model. Missing values, duplicate records, incorrect values, inconsistent formats, and outliers may cause the model to learn incorrect patterns.

As a result, the model may produce inaccurate predictions, become biased, or perform poorly on new data. Therefore, proper data preprocessing is necessary before training a machine learning model.

Q10. Name any three Python libraries commonly used for data preprocessing.

Answer:

Three commonly used Python libraries are:

1. **Pandas** – used for data cleaning, manipulation, and handling missing or duplicate data.
2. **NumPy** – used for numerical calculations and array operations.
3. **Scikit-learn** – used for preprocessing techniques such as encoding, scaling, normalization, and feature transformation.

Other useful libraries include **Matplotlib** and **Seaborn** for data visualization.